



Máster en
Inteligencia
Artificial
UMU

Deep Learning para el procesamiento de Lenguaje Natural

Tema 1. Introducción al Deep Learning.
Atención. Transformers

JUAN A. BOTÍA BLAYA
CIENCIA DE LA COMPUTACIÓN E INTELIGENCIA ARTIFICIAL
DEPT. INGENIERÍA DE LA INFORMACIÓN Y LAS COMUNICACIONES

Universidad de Murcia

Máster en Inteligencia Artificial

2026/2027





Para llegar a los transformers hay que pasar antes por el mecanismo de atención.
En otras palabras: eliminamos el bottleneck de la arquitectura encoder-decoder



¿Qué es el bottleneck de la arquitectura encoder-decoder?



Redes secuencia a secuencia

- En el tema anterior hemos visto RNNs, diseñadas para problemas en los que cada palabra generaba una salida (otra palabra, un softmax, una etiqueta)
 - En ese caso, es suficiente con usar los tokens en la cercanía del token actual
- Otros muchos problemas generan secuencias completas a partir de secuencias de entrada, no necesariamente de la misma longitud
 - Tareas como traducción, responder a preguntas, resumen de textos, diálogos...
 - En este caso, las arquitecturas encoder-decoder son más apropiadas (también denominadas redes secuencia-secuencia)



Redes secuencia a secuencia (y II)

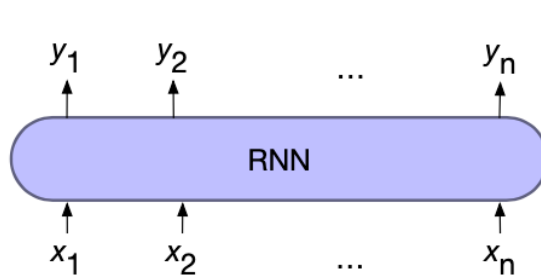
Arquitecturas encoder-decoder

- Encoder: trabaja con la secuencia en su totalidad y la codifica en forma de contexto.
 - Toma como entrada una secuencia $\langle x_1, \dots, x_n \rangle$ y genera una secuencia de estados ocultos $\langle h_1, \dots, h_n \rangle$.
 - Podemos usar LSTMs, RNNs, Transformers, etc.
- Vector de contexto: función con entrada $\langle h_1, \dots, h_n \rangle$ y salida un vector de contexto c
- Decoder: toma c como entrada
 - genera $\langle h_1, \dots, h_m \rangle$ para generar salida
 - Genera después los $\langle y_1, \dots, y_m \rangle$ de salida según la tarea específica

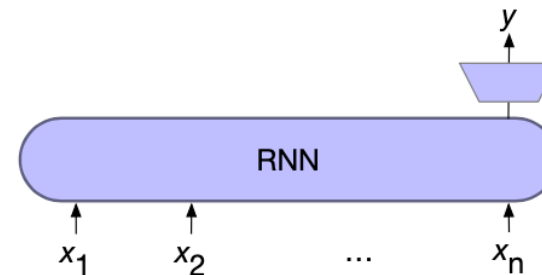
Comparativa Encoder-decoder (ED) vs. RNNs



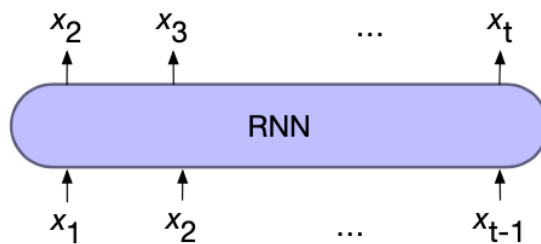
Máster en
Inteligencia
Artificial
UMU



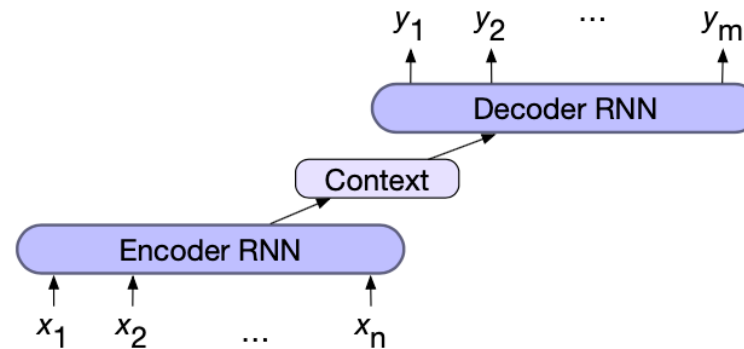
a) sequence labeling



b) sequence classification



c) language modeling



d) encoder-decoder

Fuente: cap. 8 Jurafsky et al. 2024





El problema particular de la traducción
¿Podemos atacarlo con una RNN tal cual?



El problema particular de la traducción

- Necesitamos modelar una función que tome $x = \langle x_1, x_2, \dots, x_n \rangle$ a la entrada y genere $y = \langle y_1, y_2, \dots, y_m \rangle$ a la salida
 - Hablemos solamente de la salida del decoder
 - ¿Qué queremos decir con $p(y)$ en el contexto del modelado de lenguaje?
- $$p(y) = p(y_1)p(y_2|y_1)p(y_3|y_1, y_2) \dots p(y_m|y_1, \dots, y_{m-1})$$
- Pero $p(y)$ no tiene en cuenta a x , en este problema necesitamos modelar por tanto $p(y|x)$, ¿qué forma tiene?
- Como encoder y decoder (ED) trabajan independientemente, su forma será

$$p(y|x) = p(y_1|x)p(y_2|y_1, x)p(y_3|y_1, y_2, x) \dots p(y_m|y_1, \dots, y_{m-1}, x)$$



El problema particular de la traducción (y II)

- ¿Qué ocurre con las RNNs que conocemos?
 - Generan estados ocultos h_t para cada pareja (x_t, y_t) usando h_{t-1} . Tenemos entonces las ecuaciones

$$h_t = g(h_{t-1}, x_t)$$

$$\hat{y}_t = \text{softmax}(h_t)$$

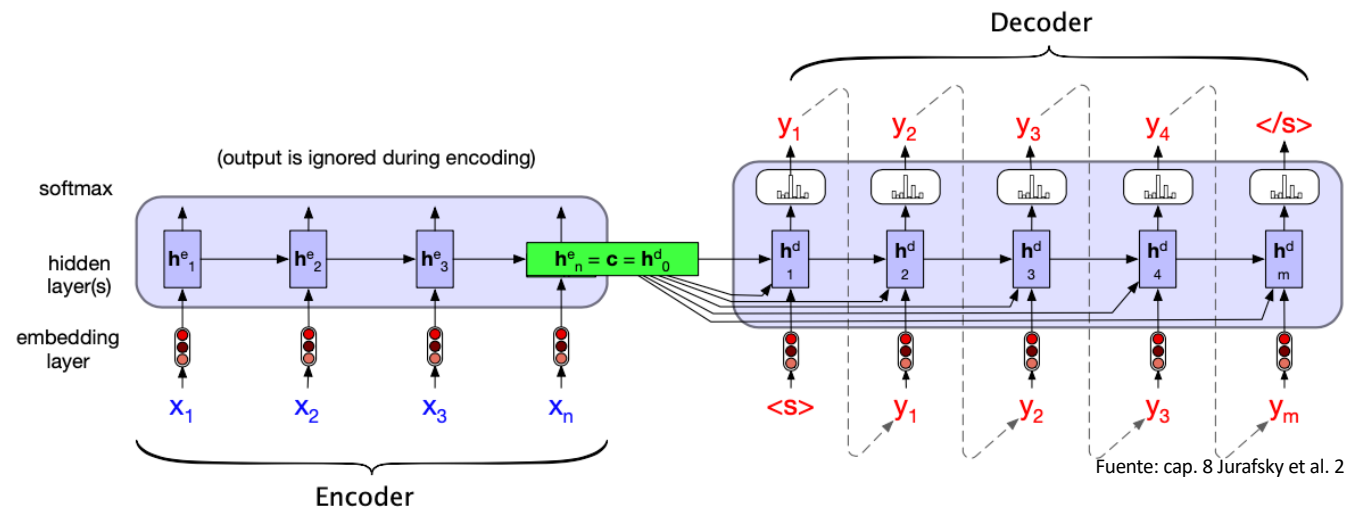
- No queremos esto, necesitamos desacoplar encoder y decoder
- Ahora nuestros ejemplos de entrenamiento serán
 - cadenas separadas por un carácter especial separador $\langle s \rangle$, como en este ejemplo
 - the green witch arrived $\langle s \rangle$ llegó la bruja verde



El problema particular de la traducción (y III)

- Procesando la Fuente: generamos estados ocultos $\langle h_1, \dots, h_n \rangle$ secuencialmente hasta procesar toda la entrada
- Procesando la salida: después, con autoregresión, desde $\langle s \rangle$ y condicionadas a los estados ocultos producidos en el decoder
- Tanto en encoder como en decoder solemos encontrar varias capas de red (e.g., biLSTMs)

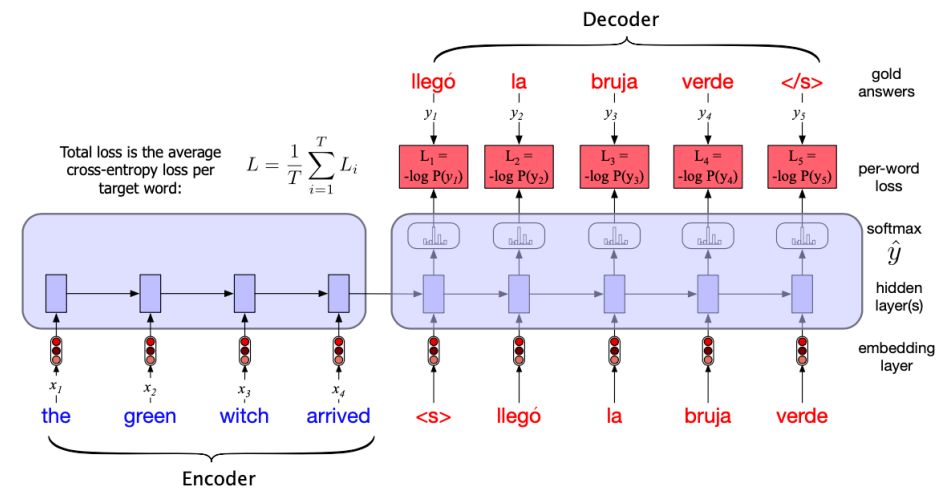
$$\begin{aligned} \mathbf{c} &= \mathbf{h}_n^e \\ \mathbf{h}_0^d &= \mathbf{c} \\ \mathbf{h}_t^d &= g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c}) \\ \hat{y}_t &= \text{softmax}(\mathbf{h}_t^d) \end{aligned}$$





Detalles sobre el entrenamiento

- En la tarea de traducción automática, los conjuntos de entrenamiento están formados por ejemplos de pares de cadenas pareadas
- A partir del separador <s>, la red se entrena de forma autoregresiva para predecir la siguiente palabra
- Se usa teacher forcing: en la autoregresión se fuerza a usar como entrada en el instante t no la palabra producida en t-1 sino la que se debería haber generado (gold target)

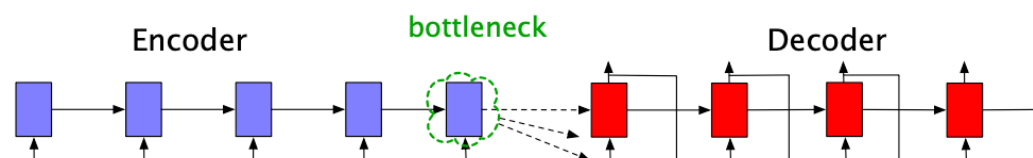


Fuente: cap. 8 Jurafsky et al. 2024



El cuello de botella en el ED

- El vector de contexto h_n a la salida del encoder debe representar toda la sentencia de entrada
 - Sesgo inevitable: tratar de forma más precisa tokens más próximos al final



Fuente: cap. 8 Jurafsky et al. 2024

- ¿Cómo podemos eliminar esa secuencialidad acumulativa de información de un estado $t-1$ al estado t y de ahí al $t+1$?
 - ¿Introduciendo algún nivel de paralelismo?
 - ¿Considerando todos los estados ocultos al mismo tiempo?



Máster en
Inteligencia
Artificial
UMU

Ahora sí: la atención para eliminar ese sesgo



2024/2025

Deep Learning para el Procesamiento de Lenguaje Natural

El mecanismo de atención

Published as a conference paper at ICLR 2015

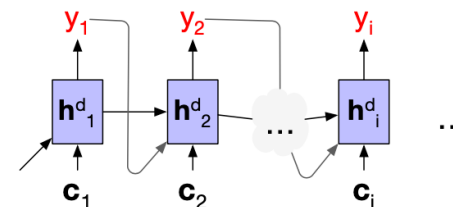
NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho Yoshua Bengio*
Université de Montréal

- Solución al cuello de botella:
 - concatenando información de todos los estados en un vector de contexto, gestionado en el decoder $\mathbf{c} = f(\mathbf{h}_1^e \dots \mathbf{h}_n^e)$
 - ponderando cada estado del encoder, según el correspondiente token del encoder sea más o menos crítico en cada momento (atención).
- El contexto $\mathbf{c}$ en el decoder pasa ahora a ser $\mathbf{c}_i$, ya que es dinámico pues la atención cambia a cada paso
- En definitiva

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$



Fuente: cap. 8 Jurafsky et al. 2024



Atención dot-product

- Dado un estado en el decoder en el instante $t-1$, h_{i-1}^d , ¿cómo de relevante es ese estado del encoder?
 - Necesitamos una f , i.e. $\text{score}(h_{i-1}^d, h_j^e)$. La atención dot product (producto escalar) es ahora

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e$$

- ¿Qué significa esto realmente?
 - Representamos estados ocultos de encoder y decoder en el mismo espacio n -dimensional de los reales
 - Productos escalares con mayor valor indican valores más altos en las mismas dimensiones (similitud de vectores además de que ambos estén alejados del origen)



Atención dot-product (y II)

- Ahora necesitamos poder comparar scores entre sí
- Dado el estado i del decoder a obtener, con α_{ij} denotamos la relevancia del estado del encoder $\mathbf{h}_j^e$ al contrastarlo con $\mathbf{h}_{i-1}^d$ y lo obtenemos con

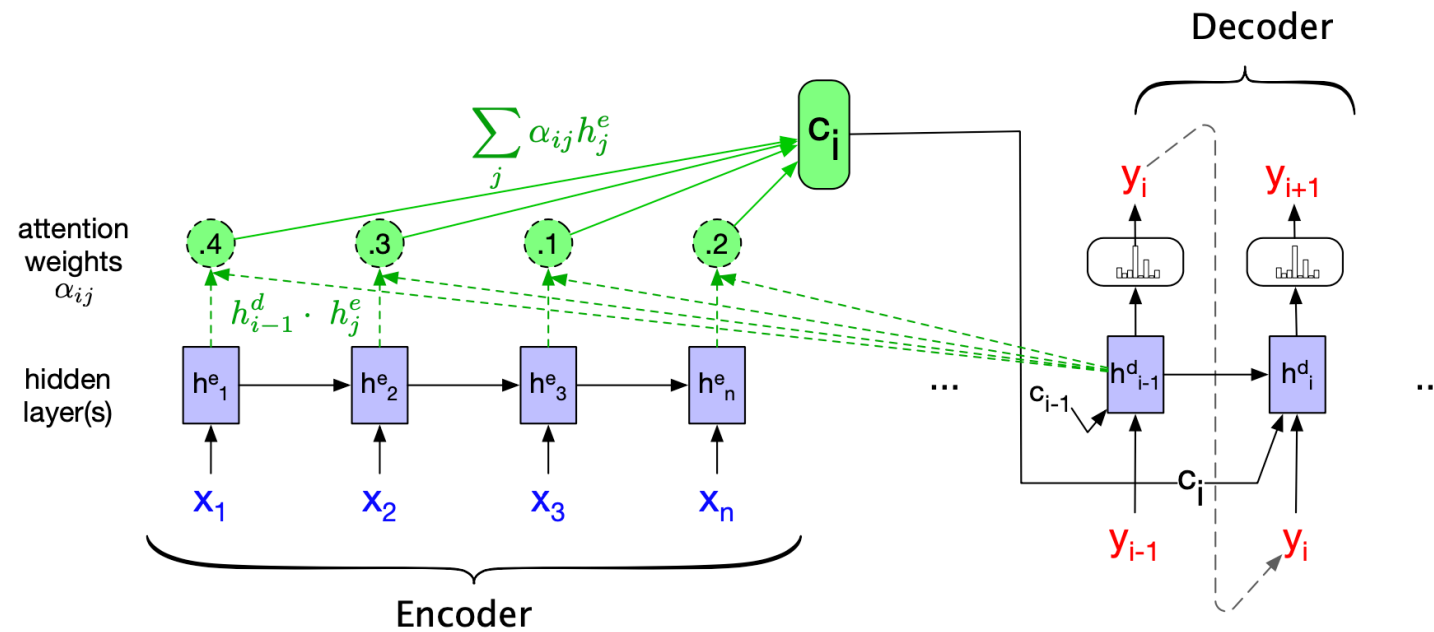
$$\begin{aligned}\alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)) \\ &= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))}\end{aligned}$$

- Así conseguimos $\mathbf{c} = \langle c_1, c_2, \dots, c_m \rangle$ para el estado actual del decoder, en el que promediamos todos los estados del encoder mediante su correspondiente atención

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$



El esquema final



Fuente: cap. 8 Jurafsky et al. 2024

Transformers!!



Máster en
Inteligencia
Artificial
UMU

- Los transformers son una arquitectura de red neuronal nueva, basada en atención, del 2017

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com



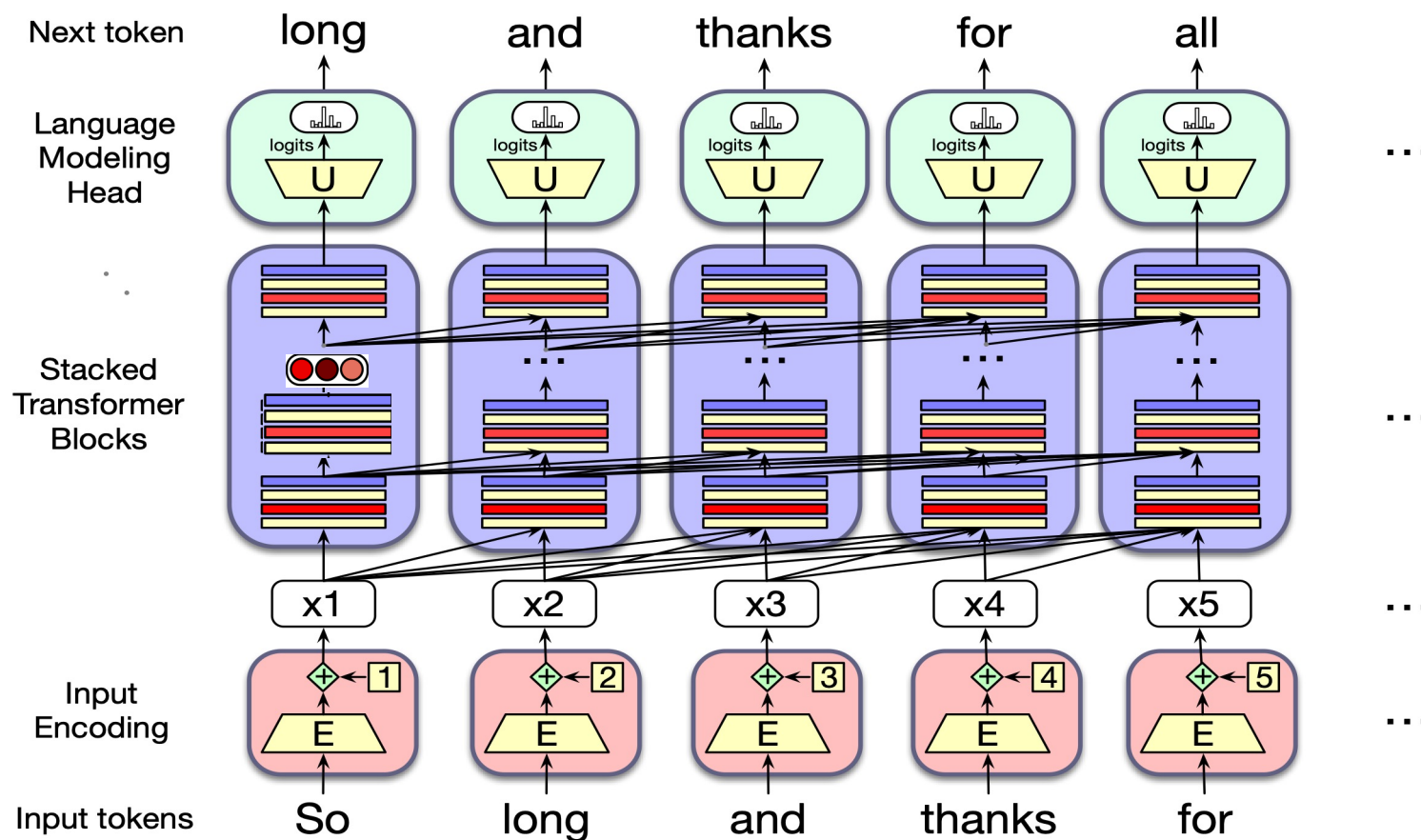
Un contexto temporal: eventos importantes para el NLP basado en IA

- 1990 Embeddings de palabras estáticos
- 2003 Modelos neuronales de lenguaje
- 2008 Aprendizaje multi-tarea
- 2015 Atención
- 2017 Transformers
- 2018 Embeddings de palabras contextuales (dinámicos) y preentrenados
- 2019 Prompting



(Auto-)Atención en transformers

Vamos a empezar viendo qué pasa usando embeddings en tokens individuales y capas individuales



2024/2025

Deep Learning para el Procesamiento de Lenguaje Natural

Los embeddings estáticos son demasiado rígidos



Máster en
Inteligencia
Artificial
UMU

- Necesitamos reflejar cómo el contexto puede cambiar el significado de las palabras
- La gallina no cruzó la calle porque **ella** estaba cansada
- ¿Cuál es el significado en un embedding estático para *ella*?



Embeddings contextuales

- Noción básica: la representación del significado de una palabra debería ser diferente en diferentes contextos!
- **Embedding contextual:** el vector de la palabra se genera en base al significado de los tokens en la cercanía
- La atención puede ser un buen mecanismo para conseguir esto!!!



Embeddings contextuales

- La gallina no cruzo la calle porque ella
- ¿Cuáles son las propiedades de *ella*?
- La gallina no cruzo la calle porque ella estaba muy cansada
- La gallina no cruzo la calle porque ella estaba oscura
- En este punto en la sentencia, o bien se refiere a la gallina o a la calle

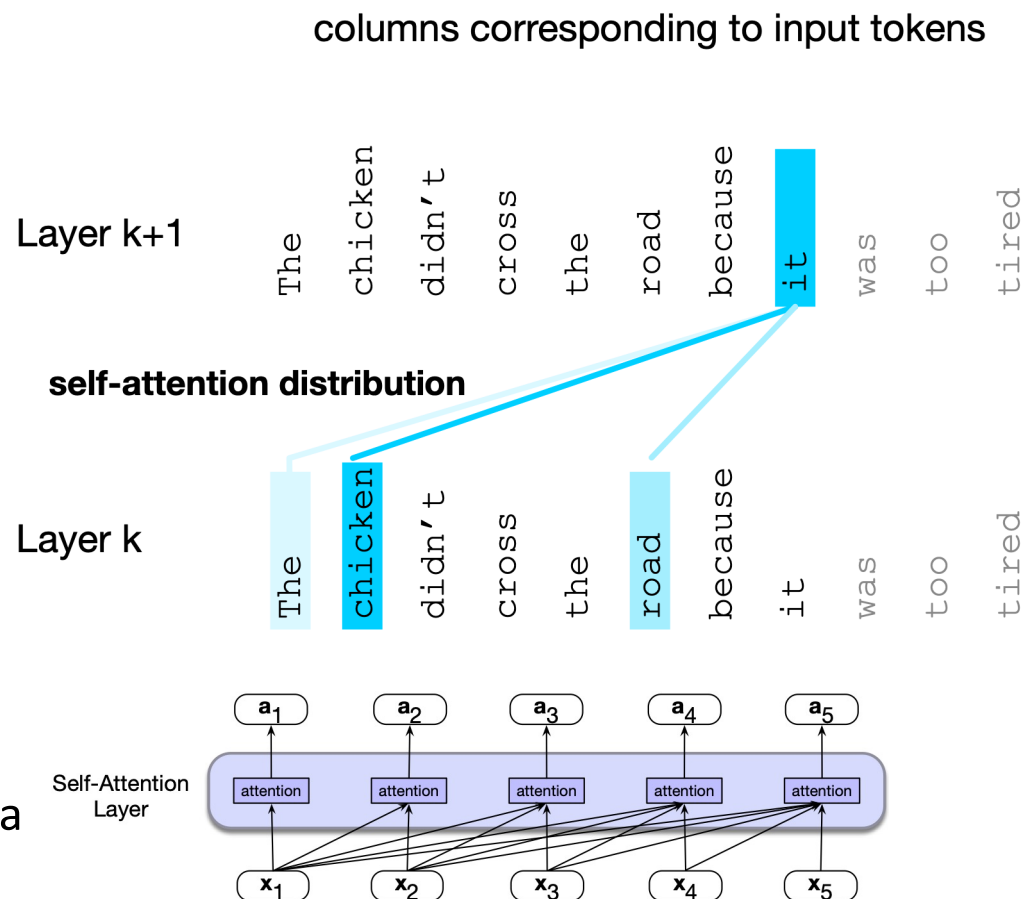
Idea básica en la noción de atención



Máster en
Inteligencia
Artificial
UMU

- Construimos el embedding contextual integrando información de las palabras vecinas de manera selectiva
- Decimos que una palabra atiende a algunas palabras vecinas más que a otras

Funciona de izquierda a derecha



Deep Learning para el Procesamiento de Lenguaje Natural

2024/2025



Atención simplificada basada en similitud



Dada una secuencia de embeddings:

$\mathbf{x}_1 \quad \mathbf{x}_2 \quad \mathbf{x}_3 \quad \mathbf{x}_4 \quad \mathbf{x}_5 \quad \mathbf{x}_6 \quad \mathbf{x}_7 \quad \mathbf{x}_i$

Generamos: $\mathbf{a}_i$ = suma ponderada de $\mathbf{x}_1$ hasta $\mathbf{x}_7$ (con $\mathbf{x}_i$)

Ponderada por su similitud con $\mathbf{x}_i$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

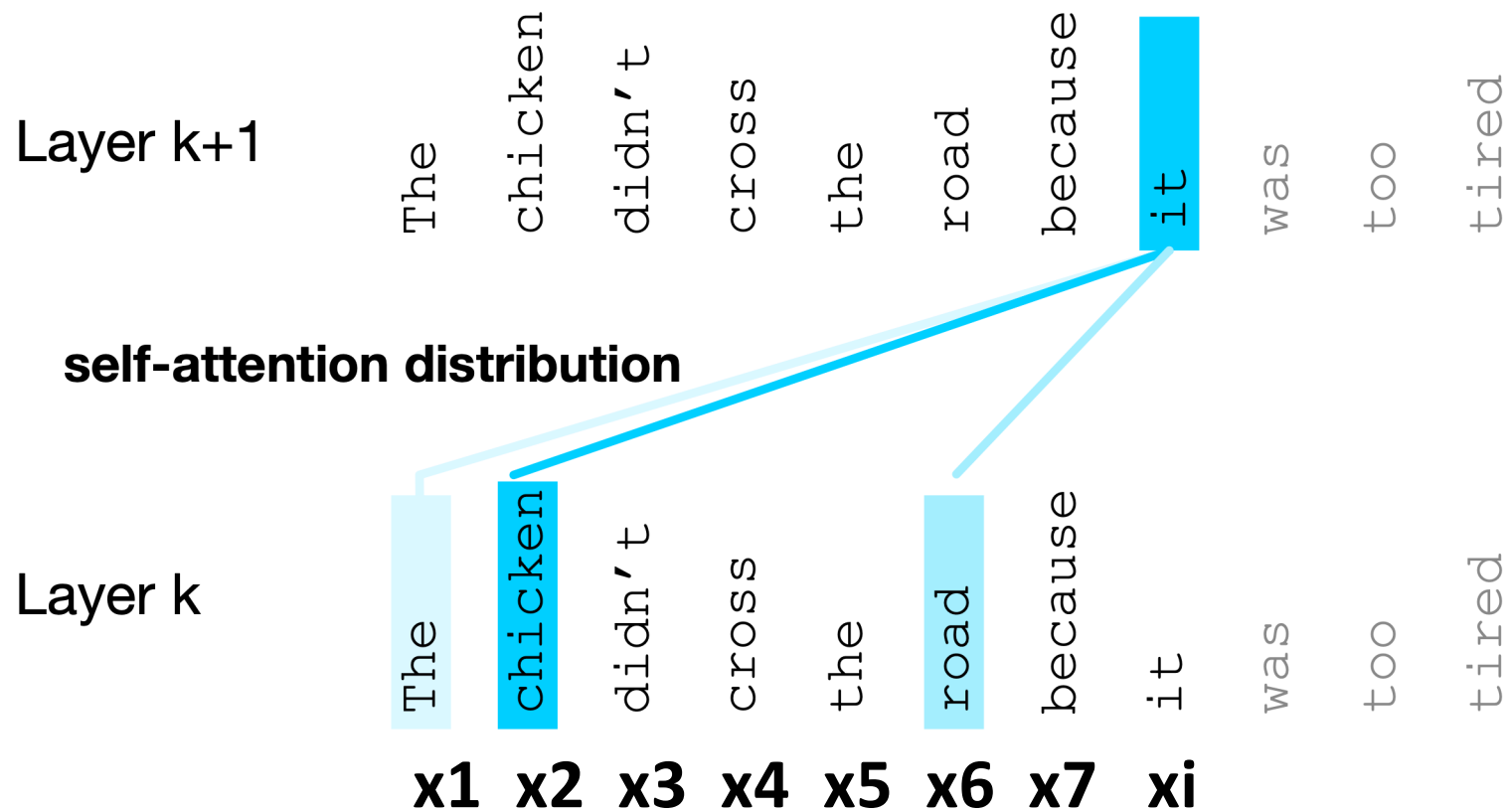
$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

Atención simplificada basada en similitud



Máster en
Inteligencia
Artificial
UMU



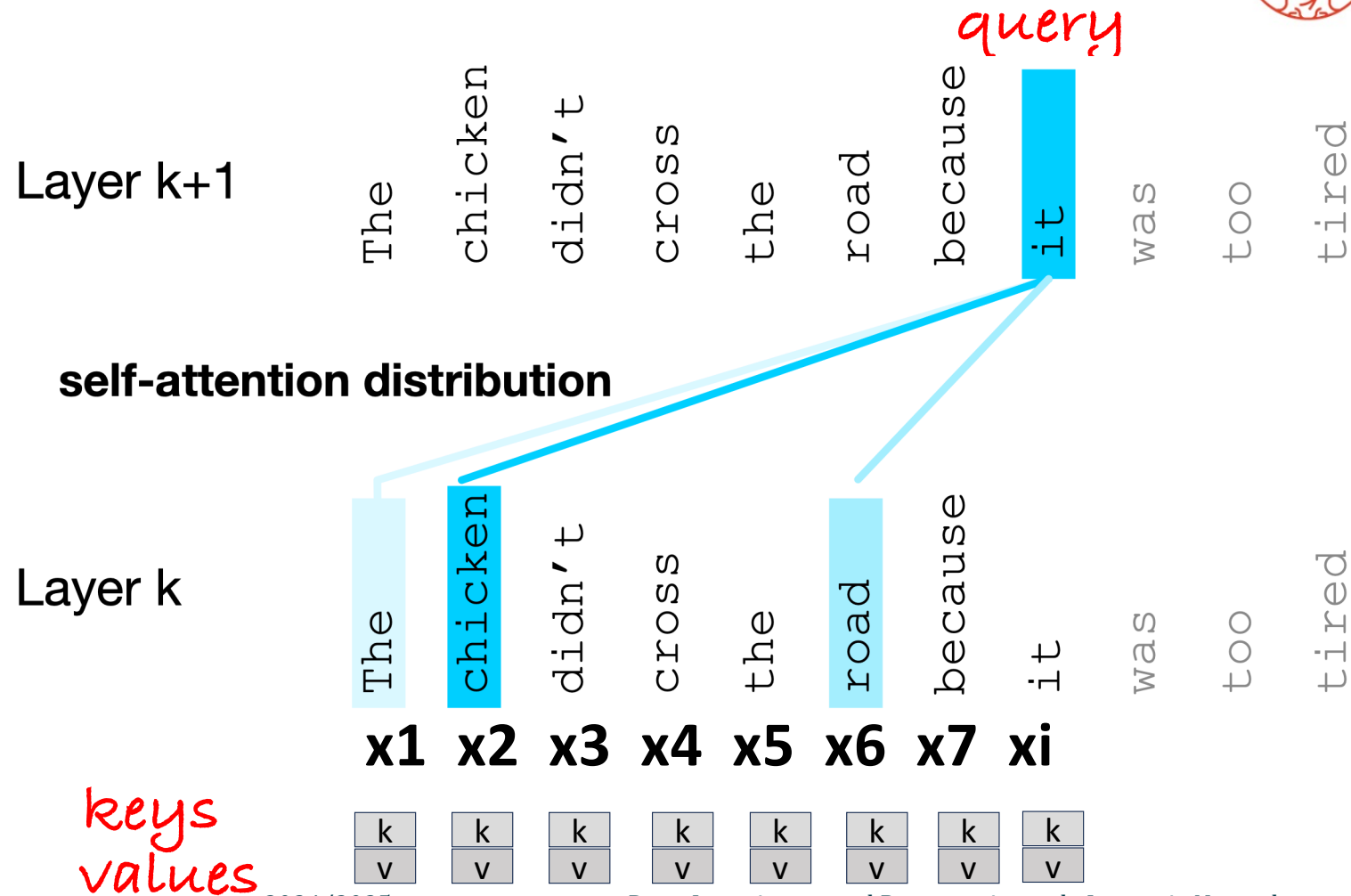
La cabeza de atención de un transformer



Máster en
Inteligencia
Artificial
UMU

- Dividimos el sistema de atención en tres elementos, con sus correspondientes vectores y pesos entrenables:
- **query**: que representa al token actual, por el cual estamos preguntando ¿en qué tokens anteriores debería de fijarme y con qué intensidad si el token actual es este?
- **key**: que representa un token anterior que estamos comparando con el actual, para obtener su similitud
- **value**: el valor de un token anterior, que ponderamos y sumamos al total

Intuition of attention:





La cabeza de atención y sus parámetros

- Según usemos el token x_i como query, key o value, su carácter se lo dará la proyección que le aplicamos mediante el producto con la matriz correspondiente a su categoría
- **query: W^Q**
- **key: W^K**
- **value: W^V**

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$



La cabeza de atención: ecuaciones finales

- Dadas las tres representaciones de $\mathbf{x}_i$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

- Calcular similitud de $\mathbf{x}_i$ con cualquier $\mathbf{x}_j$ anterior consiste en un producto interno entre $\mathbf{q}_i$ y $\mathbf{k}_j$.
- Y para agregar todos los tokens anteriores, no sumamos $\mathbf{x}_j$ sino $\mathbf{v}_j$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$$

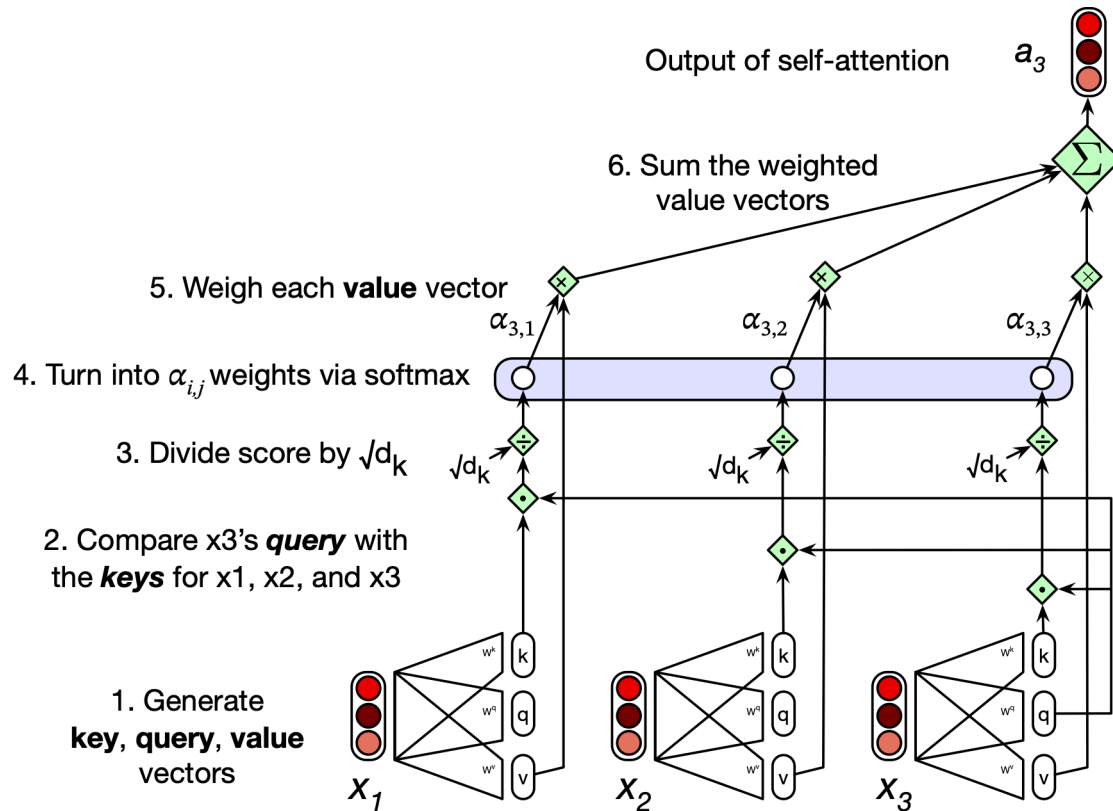
$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

La división por la raíz cuadrada de las dimensiones de los vectores query y key (d_k) de alguna forma controla que los productos internos sucesivos no generen resultados excesivamente grandes que compliquen el entrenamiento

Salida final de una cabeza para 3 tokens y embeddings de 3 dimensiones



- Tanto la entrada a una cabeza de atención, x_i , como la salida, a_i , tienen dimension $1 \times d$.
- Los vectores query y key tienen dimension $1 \times d_k$ para poder hacer $q_i \cdot k_j$.
- Los vectores valor tienen dimension d_v
- Las matrices W^q [$d \times d_k$], W^k [$d \times d_k$], W^v [$d \times d_v$]
- En el primer transformer de Vaswani, d era 512 y $d_k = d_v = 64$

En un transformer real utilizamos multiples cabezas de atención



Máster en
Inteligencia
Artificial
UMU

- La idea es que podemos estar atendiendo al contexto desde diferentes puntos de vista
- Ejemplo
 - The cat sat on the mat because it was warm
- Cabeza 1: foco en la resolución de co-referenciación: gracias a esta cabeza, podemos asociar *it* a *cat*, asumiendo que son la misma entidad
- Cabeza 2: foco en roles semánticos, identificando sujeto, *the cat*, predicado, *sat*, y objeto, *the mat*
- Cabeza 3: foco en relaciones causa-efecto, activada por *because*. Podría conectar el efecto, *the cat sat on the mat*, con la causa, *it was warm*



Otro ejemplo algo más complejo

"When the rain finally stopped, Sarah decided to take a walk in the park. The trees were still dripping with water, and the air smelled fresh. She enjoyed the peacefulness of the empty pathways, where only the sound of her footsteps echoed. Despite the lingering clouds, the sun began to peek through, casting a warm glow on the landscape."

- Cabeza 1: foco en secuencia temporal: capturando que sólo cuando termina de llover Sara decide empezar a pasear.
- Cabeza 2: descripción de la escena, poniendo el foco en los elementos adicionales como los árboles goteando, el aire oloroso y el sol comenzando a destellar
- Cabeza 3: foco en el tono emocional, centrándose en qué tipo de tono intenta transmitir mediante las palabras importantes como enjoyed, peacefulness y empty pathways, transmitiendo un momento tranquilo y reflexivo
- Cabeza 4: relaciones espaciales e interacción con objetos...
- Cabeza 5: relaciones causa efecto...



En un transformer real utilizamos multiples cabezas de atención

- Estas son las ecuaciones para hacerlo possible.
- Cada cabeza se trata en paralelo con el resto. Al final, todas se concatenan para producir una salida.

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}^c}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{K}^c}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{V}^c}; \quad \forall c \quad 1 \leq c \leq h$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

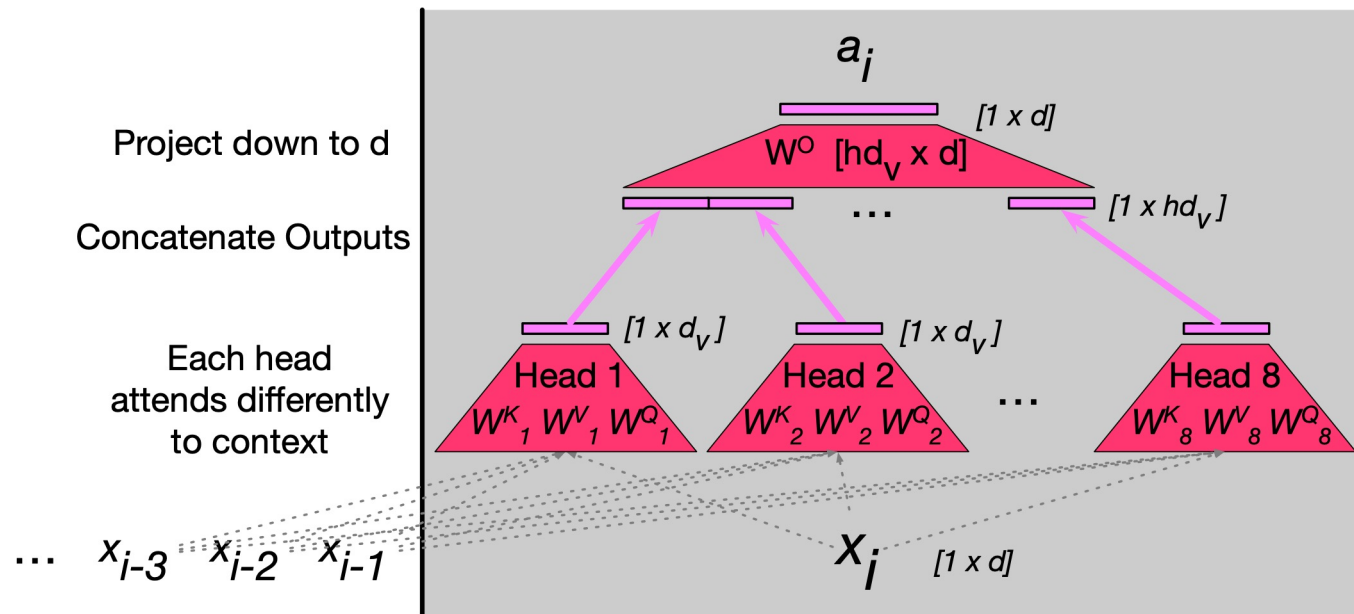
$$\text{head}_i^c = \sum_{j \leq i} \alpha_{ij}^c \mathbf{v}_j^c$$

$$\mathbf{a}_i = (\text{head}^1 \oplus \text{head}^2 \dots \oplus \text{head}^h) \mathbf{W}^O$$

$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \dots, \mathbf{x}_N]) = \mathbf{a}_i$$



Atención multicabezal



Como puede verse, un multicabezal de este tipo acepta como entrada un vector de d componentes y genera como salida otro vector de los mismos componentes. Por lo tanto, es un modulo perfectamente enchufable en un stream de datos transformador de tokens de manera secuencial, como lo que se está planteando



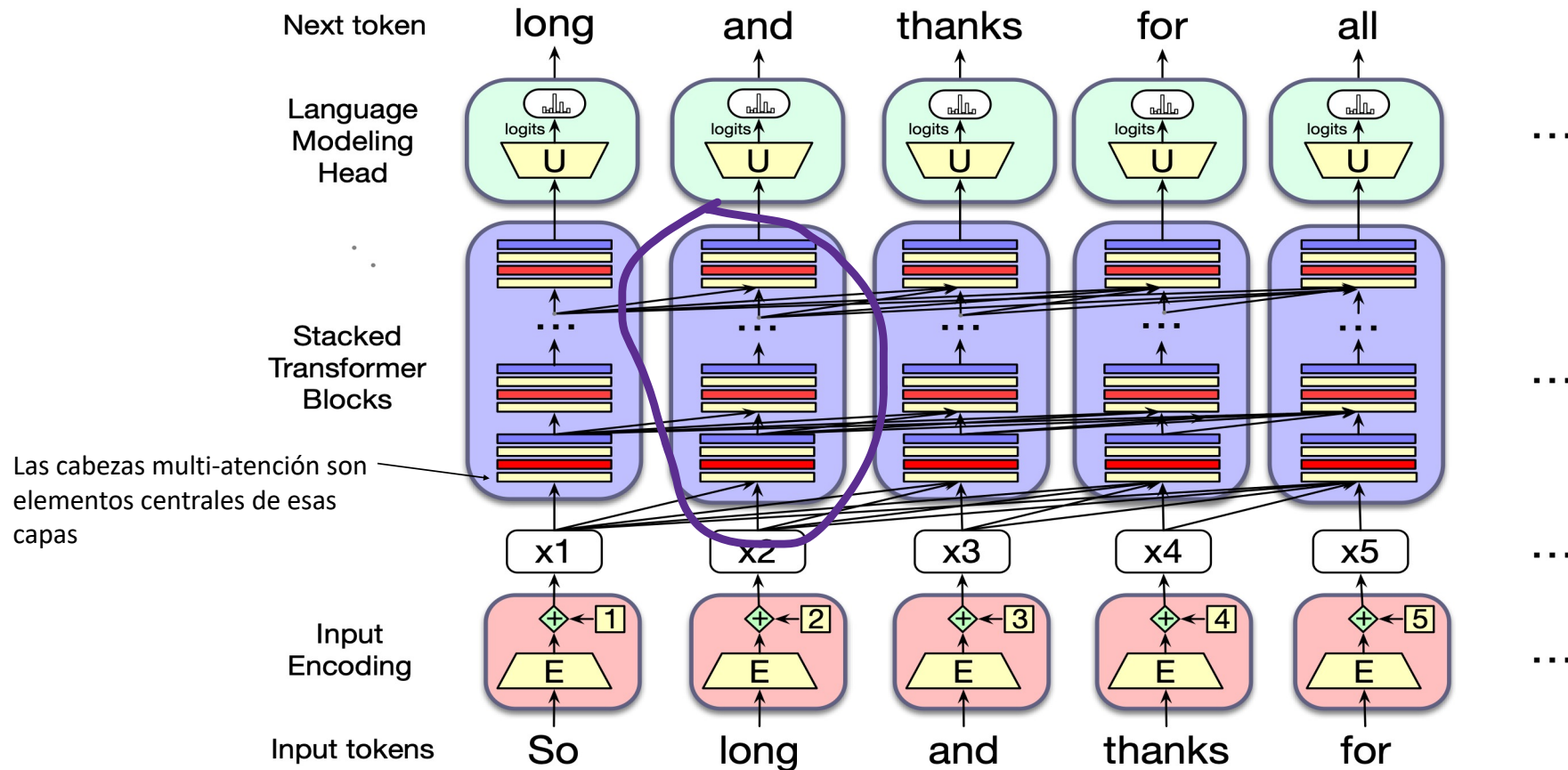
En resumen, recapitulando

- Con la atención, el contexto se hace dinámico, enriqueciendo la información que genera el encoder para cada token
- Como resultado: cada cabeza de atención va a producir un contexto diferente **según a qué se atiende**
- Embedding contextuales: representan el significado según un contexto determinado (escénico, de sentimientos, geográfico, de causa-efecto, cualquier otro que se pueda aprender de los datos)
- Veremos cómo la atención se puede usar para trasvasar información de un token a otro



El element central del transformer, sus capas transformadoras (transformer block)

Nos fijamos ahora en el elemento central de la arquitectura

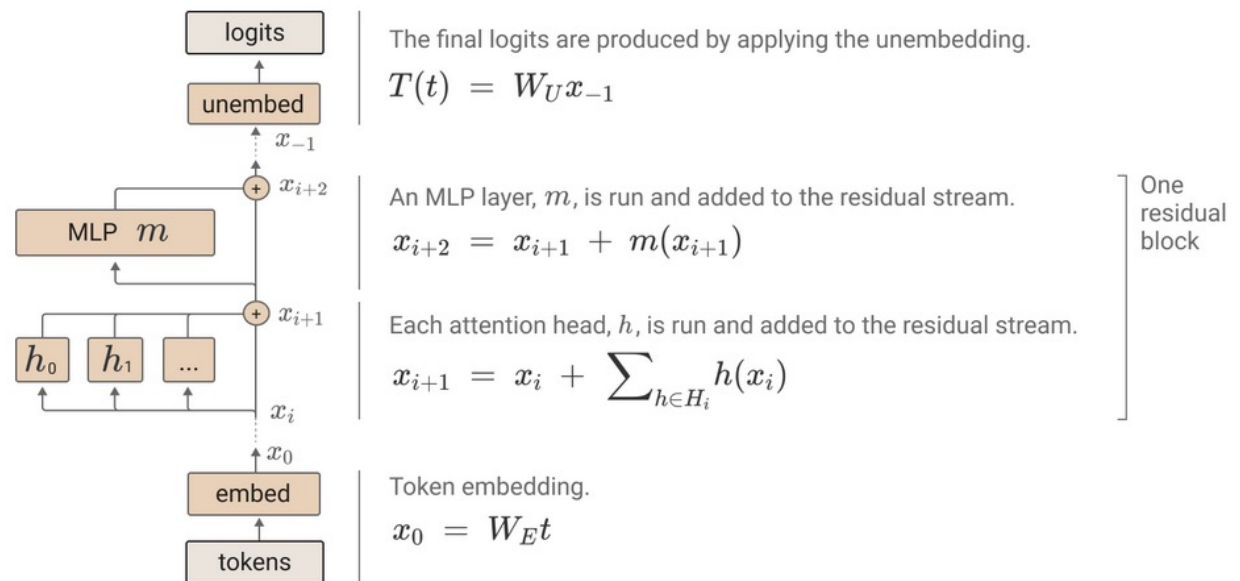


El modelo de stream residual



Máster en
Inteligencia
Artificial
UMU

- En el white paper de Anthropic se propone este modelo conceptual para ayudar a entender mejor los transformers
- El proceso comienza con el vector de entrada y cada componente lee la entrada desde el flujo residual, genera una salida y la deposita en ese mismo flujo, como en la figura

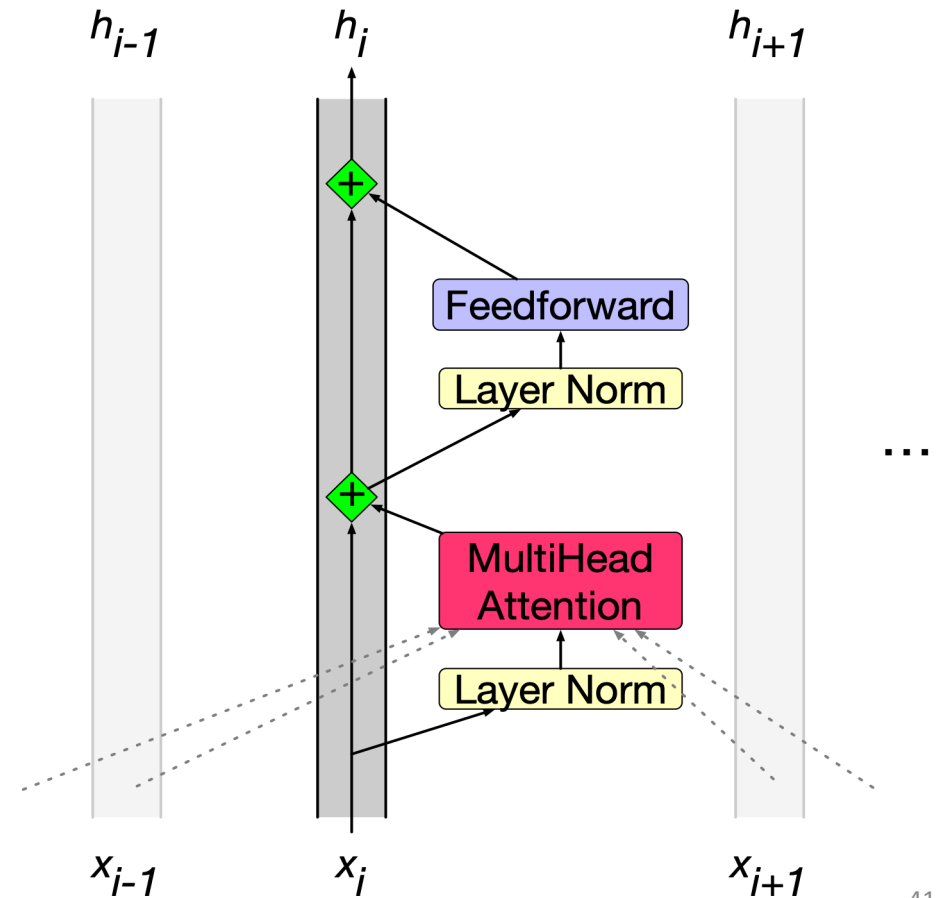


<https://transformer-circuits.pub/2021/framework/index.html>



El stream residual básico

- Comenzamos abajo con el embedding de un token, de dimensiones d
- La capa de normalización mantiene el embedding en un rango de valores adecuado para que el algoritmo SGD funcione bien
- La capa feedforward realiza un proceso no lineal de red neuronal clásica ...
- Como vemos: cada elemento lee del stream, procesa y vuelve a dejar en el stream, añadiéndolo mediante una suma. El siguiente elemento lee el resultado de esa suma y hace lo mismo





La capa feedforward

- Es un MLP con una capa oculta (dos matrices de pesos) y activación ReLU, y una capa de salida lineal
 - En la capa oculta normalmente el número de nodos es mayor que d (e.g., si $d=512$, $d_{ff}=2048$ en la capa oculta).
- Es responsable de aprender patrones no lineales que no se modelan con los otros mecanismos del transformer (e.g., embedding contextual)
- El MLP actúa poniendo el foco solamente en el token actual (en contraste con la atención), obviando el resto de tokens

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i \mathbf{W}_1 + b_1) \mathbf{W}_2 + b_2$$



La capa de normalización

- El SGD requiere de un gran número de operaciones suma y product con números positivos y negativos que puede generar problemas cuando las magnitudes de las variables crecen demasiado o se hacen muy pequeñas
- La capa de normalización mantiene los valores dentro de rangos operativos. Se aplica a la entrada y se vuelve a aplicar a la salida

- Es un z-score clásico, modulado por dos parámetros gamma y beta que se aprenden junto con el resto de parámetros

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

$$\text{LayerNorm}(\mathbf{x}) = \gamma \frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$



Y poniéndolo todo junto

- Las ecuaciones han de ejecutarse secuencialmente
- Nótese que h_i requiere de t_3 y t_5
- El stream actual incorpora información del resto de streams através de los cabezales de atención al incorporar el resto de tokens al mecanismo de atención
- Como entrada y salida mantienen la misma dimension, este bloque es apilable tanto como se quiera (e.g., T5 usa 12, GPT-4 usa 96)

$$\mathbf{t}_i^1 = \text{LayerNorm}(\mathbf{x}_i)$$

$$\mathbf{t}_i^2 = \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{x}_1^1, \dots, \mathbf{x}_N^1])$$

$$\mathbf{t}_i^3 = \mathbf{t}_i^2 + \mathbf{x}_i$$

$$\mathbf{t}_i^4 = \text{LayerNorm}(\mathbf{t}_i^3)$$

$$\mathbf{t}_i^5 = \text{FFN}(\mathbf{t}_i^4)$$

$$\mathbf{h}_i = \mathbf{t}_i^5 + \mathbf{t}_i^3$$



Input and output: Position embeddings and the Language Model Head



Embeddings de posición y de token

- La matriz X (tamaño $[N \times d]$) contiene un embedding para cada palabra del contexto.
- Cada embeddings se crea sumando dos embeddings básicos distintos
 - De significado del token
 - Y de codificación de la posición



Embedding de significado

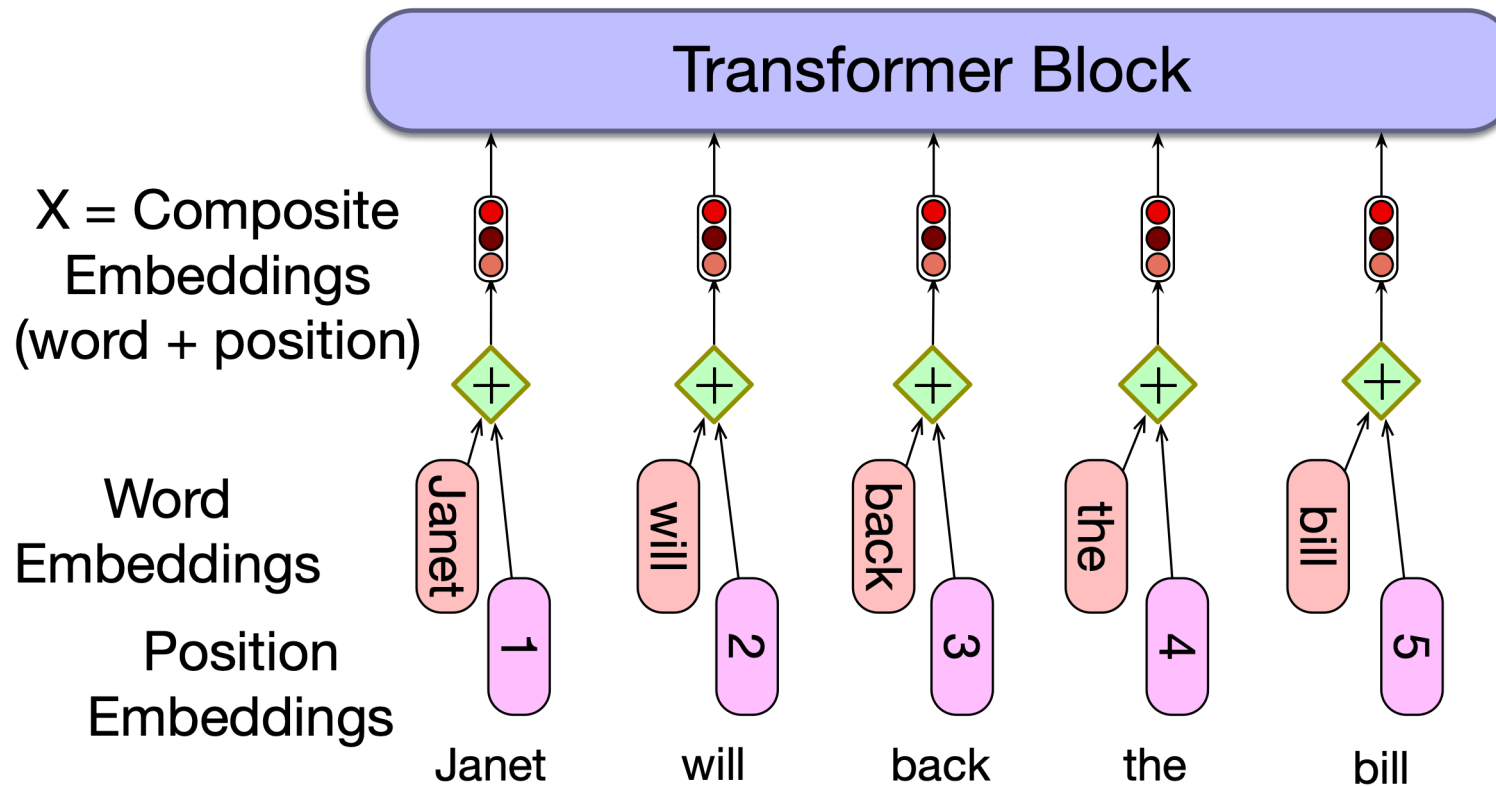
- Se parte de una matriz E de tamaño $[|V| \times d]$.
- Una fila para cada uno de los $|V|$ tokens del vocabulario
- Una columna para cada dimension de las d del embedding
- Dada la cadena "*Thanks for all the*"
- 1. *Tokenizamos y generamos sus índices en el vocabulario*
- $w = [5, 4000, 10532, 2224]$
- 2. Seleccionamos las filas de E para cada palabra
- (row 5, row 4000, row 10532, row 2224).



Codificar la posición

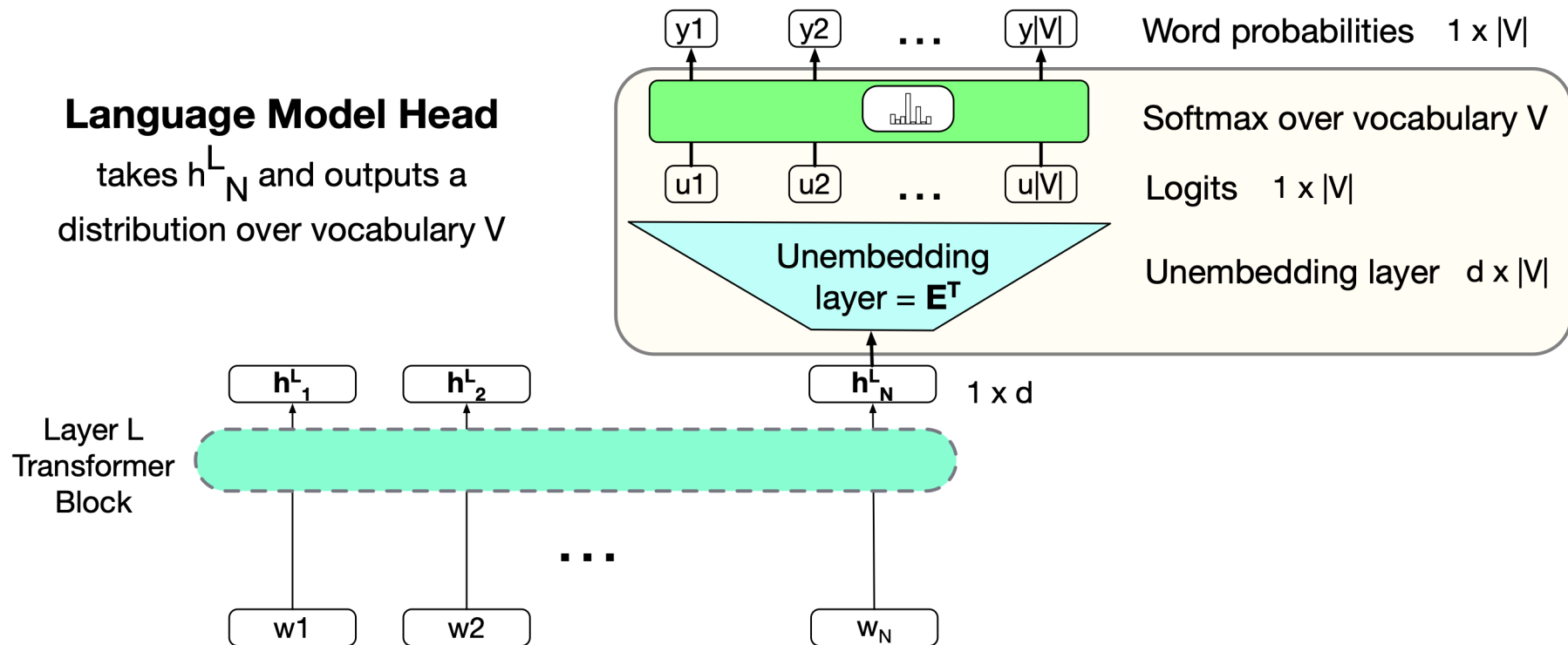
- Método de la posición absoluta (hay muchos).
- Objetivo: aprender una matriz Epos de tamaño $[1 \times N]$.
- Empezamos con un embedding random, uno para cada entero de 1 a N.
 - i.e., igual que tenemos un embedding para *fish*, ahora vamos a tener embeddings para las posiciones 3 y 17.
- Se trata de aprender a codificarlas durante el entrenamiento

Each x is just the sum of word and position embeddings

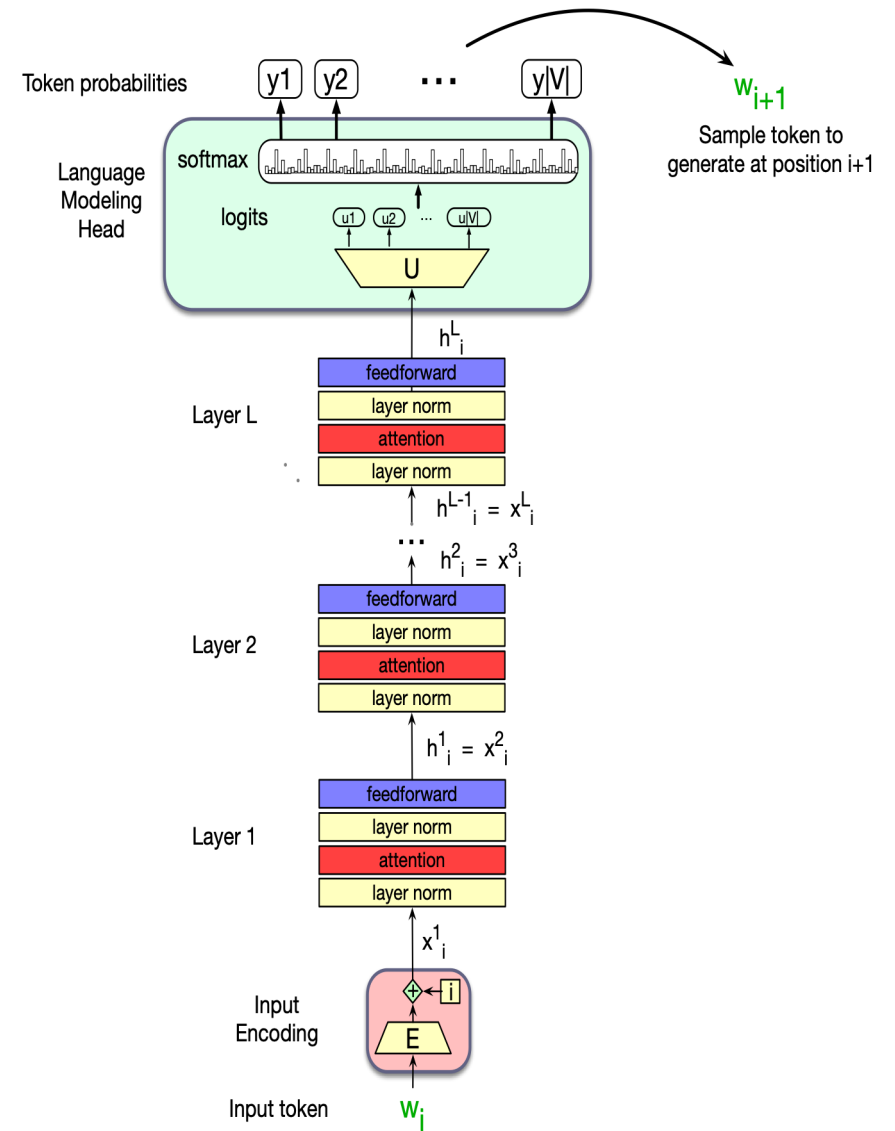
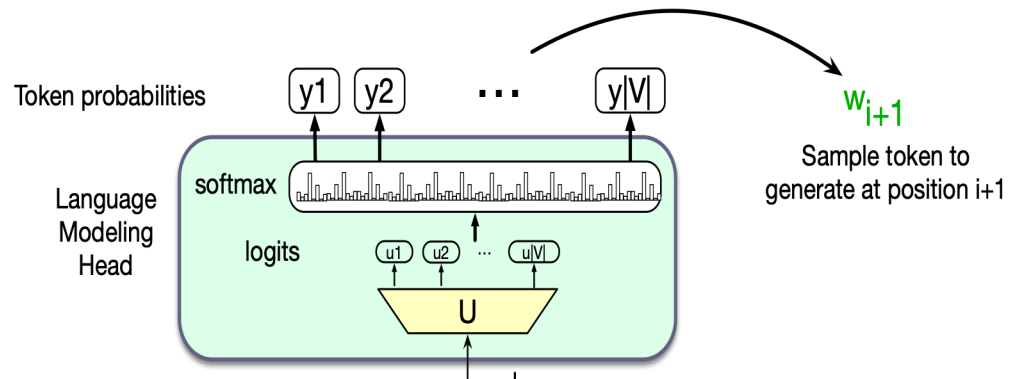




Cabezal de modelado de lenguaje



The final transformer model





Conclusiones

- Los transformers son prácticamente lineales
- Tienen una parte de encoder y otra de decoder
- Basados en atención (lineal)
- Múltiples cabezales que manejan diferentes contextos
- Codificación de tokens y posicional
- Cabeza de salida modela realmente el lenguaje